# NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY

COMPUTER SCIENCE & ENGINEERING DEPARTMENT

SECOND YEAR (3rd Semester) - 2024

DATABASE MANAGEMENT SYSTEM


**Submitted By-**

**Divyansh Ahuja (2023UCS1677)**
**Umesh Kashyap (2023UCS1693)**
**Yash Aggarwal (2023UCS1698)**

# Pharmacy Management System

## Table of Contents



---

## Introduction

### Overview

The **Pharmacy Management System (PMS)** is a web-based application designed to streamline pharmacy operations by automating key processes such as drug inventory management, order management, and customer relationship management.

The system is intended for use by pharmacy administrators to manage drugs, customers, and orders, as well as by customers to place orders and track their order history.

The PMS is built using a combination of **Streamlit** (for the frontend) and **MySQL** (for the backend database).

It is designed to be intuitive, easy to use, and scalable to meet the needs of small pharmacies.

### Objectives

- To build an automated system for managing pharmacy details, including drugs, customers, and orders.
- To create a secure user authentication system for both customers and administrators.
- To provide a responsive and intuitive interface for interacting with the database.
- To ensure the system is efficient in handling concurrent requests from multiple users.

### Technologies Used

- **Frontend**: Streamlit (Python library for creating interactive user interfaces).
- **Backend**: MySQL (Relational Database Management System).

- **Programming Language**: Python (for Streamlit and MySQL integration).
- **Web Server**: XAMPP (for running MySQL).

---

## Database Design

### Entity-Relationship Diagram (ERD)

An ER diagram is essential to visualize the structure of the database. Below is the entity-relationship diagram representing the tables and their relationships.

- **Customers**: Stores customer details.

- **Drugs**: Stores drug details.

- **DrugNames**: Stores names of the drugs.

- **Images**: Stores drug images.

- **Orders**: Stores order information linked to customers.

- **OrderItems**: Stores items in each order.

## SQL Code:

PharmacyDatabase.sql

```sql
CREATE SCHEMA PharmacyDB;
USE PharmacyDB;

CREATE TABLE IF NOT EXISTS Customers (
    Phone CHAR(10) PRIMARY KEY CHECK (Phone REGEXP '^[0-9]{10}$'),
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    City VARCHAR(50) NOT NULL,
    Password VARCHAR(100) NOT NULL
);

CREATE TABLE IF NOT EXISTS Drugs (
    ID INT PRIMARY KEY,
    ExpDate DATE NOT NULL,
    Qty INT NOT NULL,
    Price INT NOT NULL,
    D_Use VARCHAR(100) NOT NULL
);

CREATE TABLE IF NOT EXISTS DrugNames (
    ID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    FOREIGN KEY (ID) REFERENCES Drugs(ID) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS Images (
    ID INT PRIMARY KEY,
    Image LONGBLOB,
    FOREIGN KEY (ID) REFERENCES Drugs(ID) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS Orders (
    O_ID VARCHAR(100) PRIMARY KEY,
    PhoneNumber CHAR(10) NOT NULL,
    FOREIGN KEY(PhoneNumber) REFERENCES Customers(Phone) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS OrderItems (
    O_ID VARCHAR(100),
```

```sql
CREATE TABLE IF NOT EXISTS OrderItems (
    O_ID VARCHAR(100),
    Item_Name VARCHAR(100),
    Item_Qty INT NOT NULL,
    PRIMARY KEY (O_ID, Item_Name),
    FOREIGN KEY (O_ID) REFERENCES Orders(O_ID) ON DELETE CASCADE
);

```

## Normalization Process

- **1NF**: All attributes contain atomic values. There are no repeating groups or arrays within the tables. No multivalued attributes are used in the entire database.

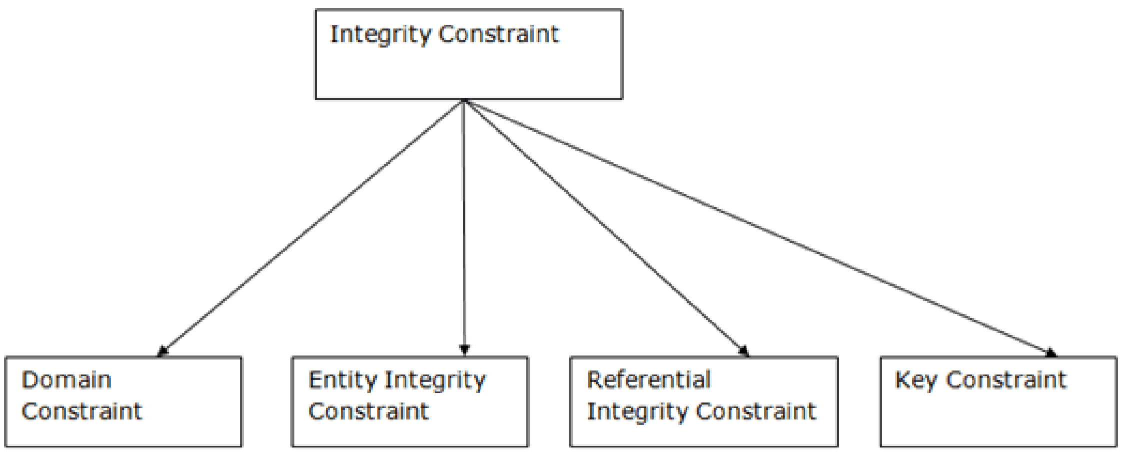
- **2NF**: The tables do not contain partial dependencies. All non-key attributes depend fully on the primary key.

- For example, in the drugs relation, DrugName and ID were both candidate keys and hence resulted in Partial Dependency. So, we broke the Drug table into 2 parts: DrugNames and other containing Drug details.

- **3NF**: There are no transitive dependencies. Non-key attributes depend only on the primary key.

---

## Integrity constraints:




### 1. Domain Constraints:

For all the relations used, we have defined the datatypes of the attributes.

Also in the Customers table a check has been defined on the phone attribute that it can be of exactly 10 digits.

```
Phone CHAR(10) PRIMARY KEY CHECK (Phone REGEXP '^[0-9]{10}$'),
```

## 2. Entity Integrity Constraints:

The entity integrity constraint states that primary key value can't be null.

In the project, we have defined that all the primary keys must be non null.

## 3. Referential Integrity Constraints:

This defines the primary key foreign key relationships in the system.

```
CREATE TABLE IF NOT EXISTS DrugNames (
    ID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    FOREIGN KEY (ID) REFERENCES Drugs(ID) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS Images (
    ID INT PRIMARY KEY,
    Image LONGBLOB,
    FOREIGN KEY (ID) REFERENCES Drugs(ID) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS Orders (
    O_ID VARCHAR(100) PRIMARY KEY,
    PhoneNumber CHAR(10) NOT NULL,
    FOREIGN KEY(PhoneNumber) REFERENCES Customers(Phone) ON DELETE CASCADE
);
```

In DrugName ID is the foreign key that references the drugs table.

In Images ID is the foreign key that references the drugs table.

In Orders PhoneNumber is the foreign key that references the Customer table.

4. Key Constraints:

Keys are the entity set that is used to identify an entity within its entity set uniquely.

In all the relations, we define at least one key to be the primary key, thus incorporating the key constraints.

## 3. Implementation/Code

main.py > ...

```python
import streamlit as st
import pandas as pd
from PIL import Image
import random
import mysql.connector
import io

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="PharmacyDB"
)

if conn.is_connected():
    print("Success")

c = conn.cursor()


def customer_add_data(phone, firstName, lastName, city, password):
    c.execute("INSERT INTO Customers (Phone, FirstName, LastName, City, Password) VALUES (%s, %s, %s, %s, %s)",
                (phone, firstName, lastName, city, password))
    conn.commit()

def customer_view_all_data():
    c.execute('SELECT * FROM Customers')
    customer_data = c.fetchall()
    return customer_data

def customer_delete(phone):
    c.execute(''' DELETE FROM Customers WHERE Phone = %s''', (phone,))
    conn.commit()

def drug_update(Dqty, Did):
    c.execute(''' UPDATE Drugs SET Qty = %s WHERE ID = %s''', (Dqty,Did))
    conn.commit()

def drug_delete(Did):
```

Ln 12, Col 26 Spaces: 4

```python
def drug_delete(Did):
    c.execute(''' DELETE FROM Drugs WHERE ID = %s''', (Did,))
    conn.commit()

def drug_add_data(Did, Dname, Dexpdate, Duse, Dqty, DPrice):
    c.execute('''INSERT INTO Drugs (ID, ExpDate, Qty, Price, D_Use) VALUES(%s,%s,%s,%s, %s)''', (Did, Dexpdate, Dqty, DPrice, Duse))
    c.execute('''INSERT INTO DrugNames (ID, Name) VALUES(%s,%s)''', (Did, Dname))
    conn.commit()

def drug_view_all_data():
    c.execute('SELECT * FROM DrugNames NATURAL JOIN Drugs')
    drug_data = c.fetchall()
    return drug_data

def order_delete(Oid):
    c.execute(''' DELETE FROM Orders WHERE O_ID = %s''', (Oid,))

def order_add_data(phoneNumber,O_Items,O_Qty,O_id):
    c.execute('''INSERT INTO Orders (O_ID, PhoneNumber) VALUES(%s,%s)''',
            (O_id, phoneNumber))
    O_Qty = O_Qty[0:len(O_Qty)-1]
    O_Items = O_Items[0:len(O_Items)-1]
```

```python
    qtyList = O_Qty.split(",")
    itemsList = O_Items.split(",")
    for i in range(len(qtyList)):
        c.execute("INSERT INTO OrderItems (O_ID, Item_Name, Item_Qty) VALUES(%s, %s, %s)", (O_id, itemsList[i], qtyList[i]))
    conn.commit()

def order_view_data(phone):
    c.execute('SELECT O_ID, Item_Name, Item_Qty  FROM Orders NATURAL JOIN OrderItems Where PhoneNumber = %s',(phone,))
    order_data = c.fetchall()
    return order_data

def order_view_all_data():
    c.execute('SELECT O_ID, FirstName, LastName, Phone, City, Item_Name, Item_Qty FROM Orders NATURAL JOIN OrderItems NATURAL JOIN Cus
    order_all_data = c.fetchall()
    return order_all_data

def admin():
    st.title("Your Pharmacy Dashboard")
    menu = ["Drugs", "Customers", "Orders", "About"]
    choice = st.sidebar.selectbox("Menu", menu)

    if choice == "Drugs":
        menu = ["Add", "View", "Update", "Delete"]
        choice = st.sidebar.selectbox("Menu", menu)
        if choice == "Add":
            st.subheader("Add Drugs")
            col1, col2 = st.columns(2)
            with col1:
                drug_name = st.text_area("Enter the Drug Name")
                drug_expiry = st.date_input("Expiry Date of Drug (YYYY-MM-DD)")
                drug_mainuse = st.text_area("When to Use")
                uploaded_file = st.file_uploader("Choose an image...", type=["png", "jpg", "jpeg"])
            with col2:
                drug_quantity = st.text_area("Enter the quantity")
                drug_price = st.text_area("Enter the price")
                drug_id = st.text_area("Enter the Drug id (example: 1)")
            if st.button("Add Drug"):
```

```python
                if uploaded_file is not None:
                    image_data = uploaded_file.read()
                    drug_add_data(drug_id, drug_name, drug_expiry, drug_mainuse, drug_quantity, drug_price)
                    c.execute("INSERT INTO Images (ID, Image) VALUES (%s, %s)", (drug_id, image_data,))
                    st.success("Successfully Added Data")
                    conn.commit()

        if choice == "View":
            st.subheader("Drug Details")
            drug_result = drug_view_all_data()
            print(drug_result)
            with st.expander("View All Drug Data"):
                drug_clean_df = pd.DataFrame(drug_result, columns=["ID", "Name", "Expiry Date", "Quantity", "Price", "Use"])
                st.dataframe(drug_clean_df)

        if choice == 'Update':
            st.subheader("Update Drug Details")
            d_id = st.text_area("Drug ID")
            d_qty = st.text_area("Enter the new Quantity")
            if st.button(label='Update'):
                if(int(d_qty) >= 0):
                    drug_update(int(d_qty),int(d_id))
                    st.success("Updated Quantity Successfully!")
                else:
                    st.error("Please enter a positive value")


        if choice == 'Delete':
            st.subheader("Delete Drugs")
            did = st.text_area("Enter drug ID:")
            if st.button(label="Delete"):
                drug_delete(int(did))
                st.success("Deleted successfully!")

    elif choice == "Customers":
        menu = ["View", "Delete"]
        choice = st.sidebar.selectbox("Menu", menu)
```

```python
        if choice == "View":
            st.subheader("Customer Details")
            cust_result = customer_view_all_data()
            with st.expander("View All Customer Data"):
                cust_clean_df = pd.DataFrame(cust_result, columns=["Phone Number", "First Name", "Last Name", "City" , "Password"])
                st.dataframe(cust_clean_df)

        if choice == 'Delete':
            st.subheader("Delete Customer")
            cust_ph = st.text_area("Enter the phone number of the customer to be deleted:")
            if st.button(label="Delete"):
                customer_delete(cust_ph)
                st.success("Deleted Customer successfully!")

    elif choice == "Orders":
        menu = ["View"]
        choice = st.sidebar.selectbox("Menu", menu)
        if choice == "View":
            st.subheader("Order Details")
            order_result = order_view_all_data()
            with st.expander("View All Order Data"):
                order_clean_df = pd.DataFrame(order_result, columns=["Order ID", "First Name", "Last Name", "Phone Number", "City", "I
                st.dataframe(order_clean_df)

    elif choice == "About":
        st.subheader("DBMS Project: Pharmacy Management System")
        st.subheader("Umesh Kashyap (2023UCS1693)")
        st.subheader("Yash Aggarwal (2023UCS1698)")
        st.subheader("Divyansh Ahuja (2023UCS1677)")

def getauthenicate(phone, password):
    c.execute('SELECT Password FROM Customers WHERE Phone = %s', (phone,))
    cust_password = c.fetchall()
    if(len(cust_password) == 0):
        return False
    if cust_password[0][0] == password:
        return True
    else:
```

main.py > ...

```python
def customer(phone, password):
    if getauthenicate(phone, password):
        st.title("Welcome to Pharmacy Store")

        st.subheader("Your Order Details")
        order_result = order_view_data(phone)
        with st.expander("View All Order Data"):
            order_clean_df = pd.DataFrame(order_result, columns=["ID", "Items", "Qty"])
            st.dataframe(order_clean_df)

        drug_result = drug_view_all_data()
        print(drug_result)
        O_List = []

        for i in range(len(drug_result)):
            st.subheader("Drug: "+ str(drug_result[i][1]))
            c.execute("SELECT image FROM images WHERE id=%s", (drug_result[i][0],))
            result = c.fetchone()
            st.image(Image.open(io.BytesIO(result[0])), caption="Price Rs." + str(drug_result[i][4]), width=150)
            medSlider = st.slider(label="Quantity",min_value=0, max_value=100, key = (10*(i**2) + 1000))
            O_List.append(int(medSlider))

            st.info("When to USE: " + str(drug_result[i][5]))

        if st.button(label="Buy now"):
            O_Qty = ""
            O_Items = ""
            flag = 1
            for i in range(len(drug_result)):
                if(O_List[i] > 0):
                    c.execute('SELECT Qty FROM Drugs WHERE ID = %s', (drug_result[i][0],))
                    current_qty = c.fetchone()[0]
                    if(current_qty < O_List[i]):
                        O_List = []
                        flag = 0
                        st.error("Not enough stock")
                        break
                    O_Qty += (str(O_List[i]) + str(','))
                    O_Items += (drug_result[i][1] + ",")
```

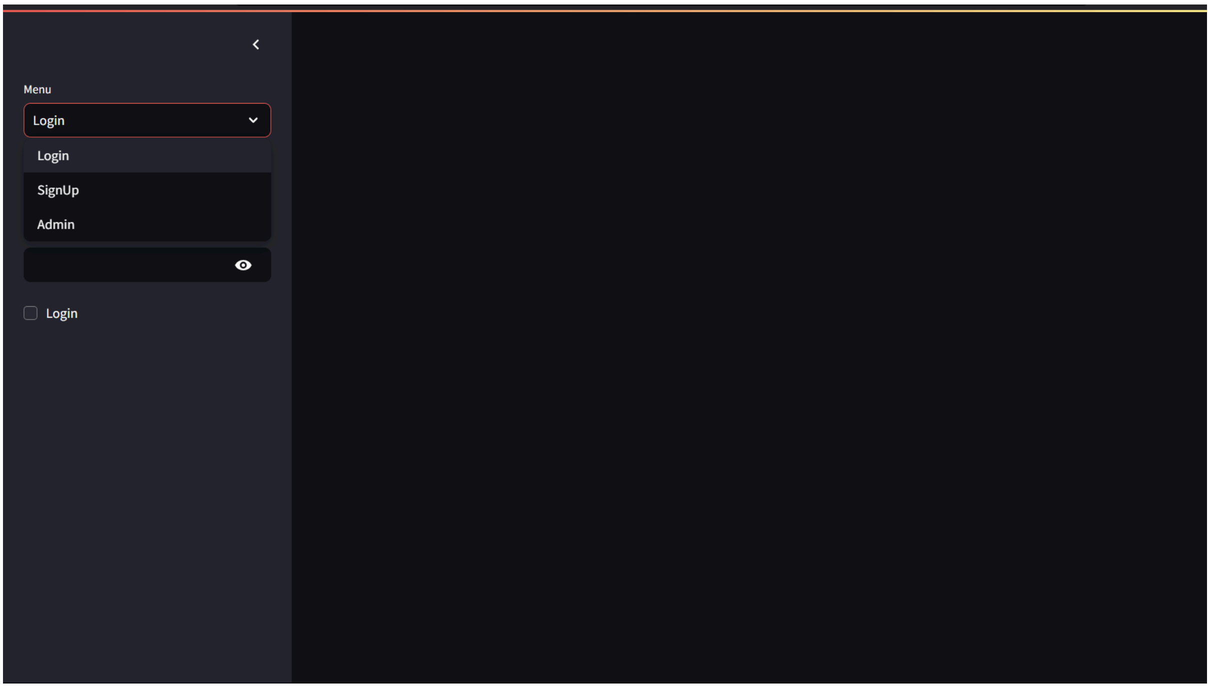
```python
def customer(phone, password):

            if(flag == 1 and set(O_List) != {0}):
                O_id = "#O" + str(random.randint(0,1000000))
                order_add_data(phone, O_Items, O_Qty, O_id)
                for i in range(len(O_List)):
                    if O_List[i] > 0:
                        c.execute('SELECT Qty FROM Drugs WHERE ID = %s', (drug_result[i][0],))
                        current_qty = c.fetchone()[0]
                        print(current_qty)
                        new_qty = current_qty - O_List[i]
                        print(new_qty)


                # Update the database with the new quantity
                        c.execute('''UPDATE Drugs SET Qty = %s WHERE ID = %s''', (new_qty, drug_result[i][0],))
                conn.commit()

                st.success("Order placed successfully! Please refresh the page")
                # c.execute(''' UPDATE Drugs SET C_Number = %s WHERE C_ID = %s''', (Cnumber,Cid))

            if(set(O_List) == {0}):
                st.error("Please enter some items before placing an order")
    else:
        st.error("Wrong credentials")


if __name__ == '__main__':
    menu = ["Login", "SignUp","Admin"]
    choice = st.sidebar.selectbox("Menu", menu)
    if choice == "Login":
        username = st.sidebar.text_input("Registered Phone Number:")
        password = st.sidebar.text_input("Password", type='password')
        if st.sidebar.checkbox(label="Login"):
            customer(username, password)

    elif choice == "SignUp":
        st.subheader("Create New Account")
        cust_fname = st.text_input("First Name")
```

Ln 12, Col 26 Spaces: 4

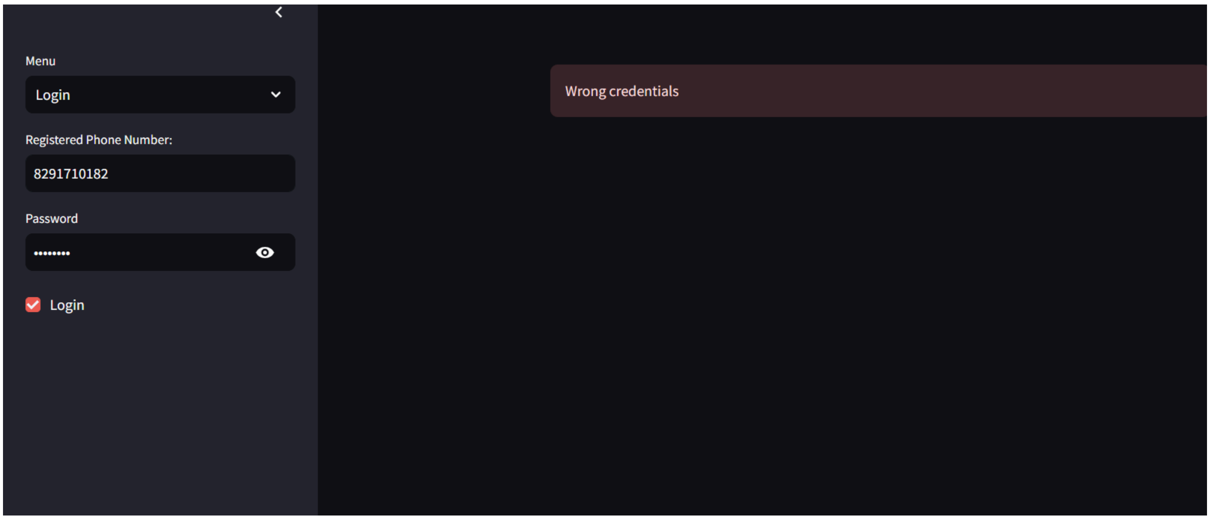
```python
        cust_lname = st.text_input("Last Name")
        cust_password = st.text_input("Password", type='password', key=1000)
        cust_password1 = st.text_input("Confirm Password", type='password', key=1001)
        col1, col2 = st.columns(2)

        with col1:
            cust_area = st.text_area("City")
        with col2:
            cust_number = st.text_area("Phone Number")

        if st.button("Signup"):
            if (cust_password == cust_password1):
                customer_add_data(cust_number, cust_fname, cust_lname, cust_area, cust_password,)
                st.success("Account Created!")
                st.info("Go to Login Menu to login")
            else:
                st.warning('Password dont match')
     elif choice == "Admin":
        username = st.sidebar.text_input("User Name")
        password = st.sidebar.text_input("Password", type='password')
        if st.sidebar.checkbox(label="Login"):
            if username == 'admin' and password == 'admin':
                admin()
            else:
                st.error("Wrong credentials")
```

# 4. Sample Output/Runs

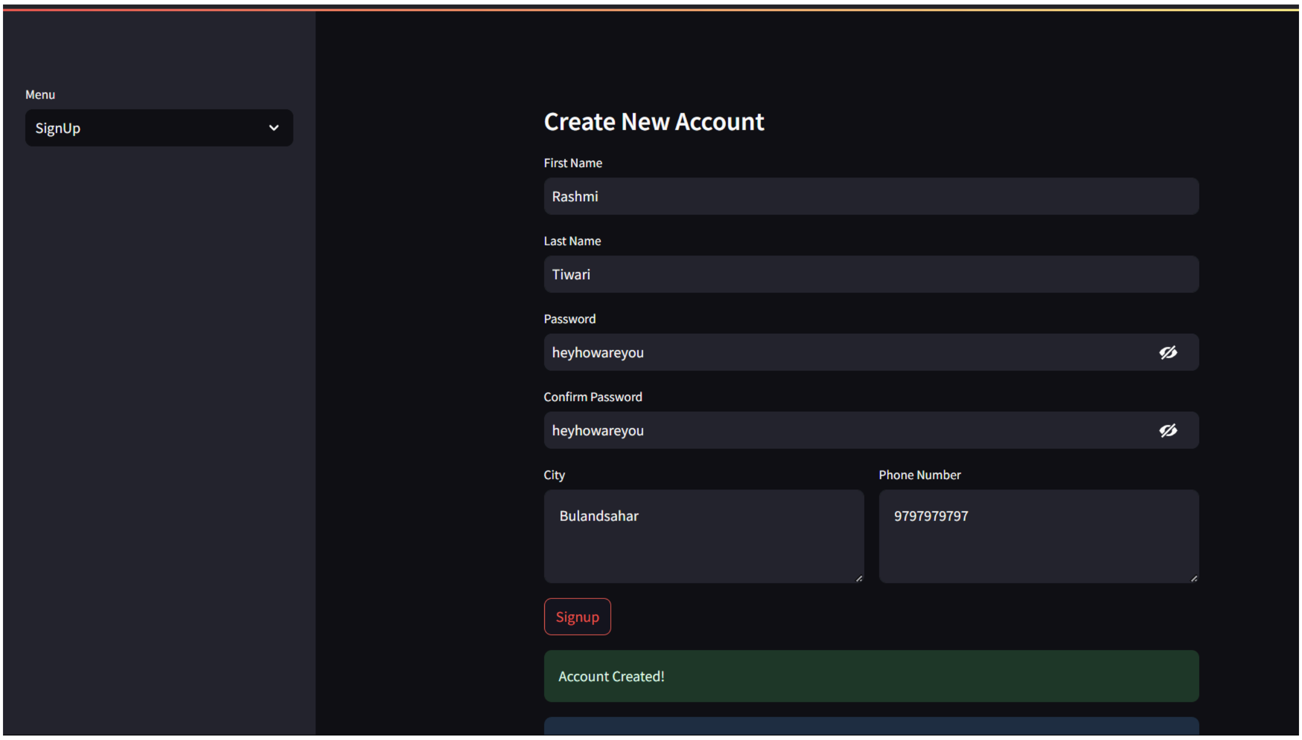
## Dashboard to choose mode:

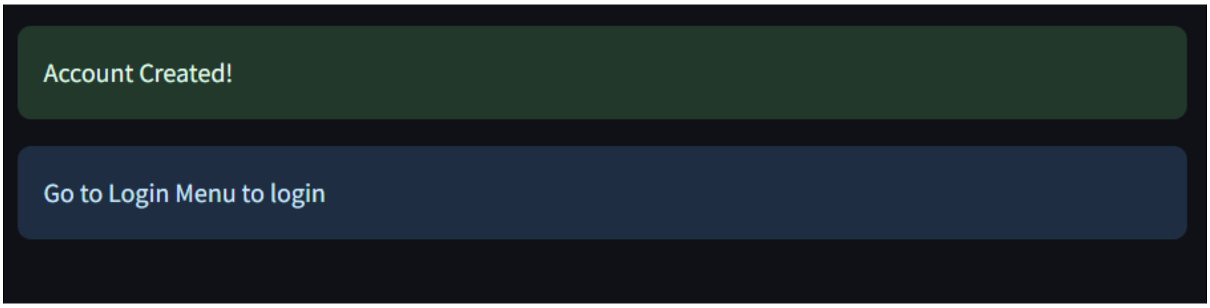



## Error if wrong credentials entered:

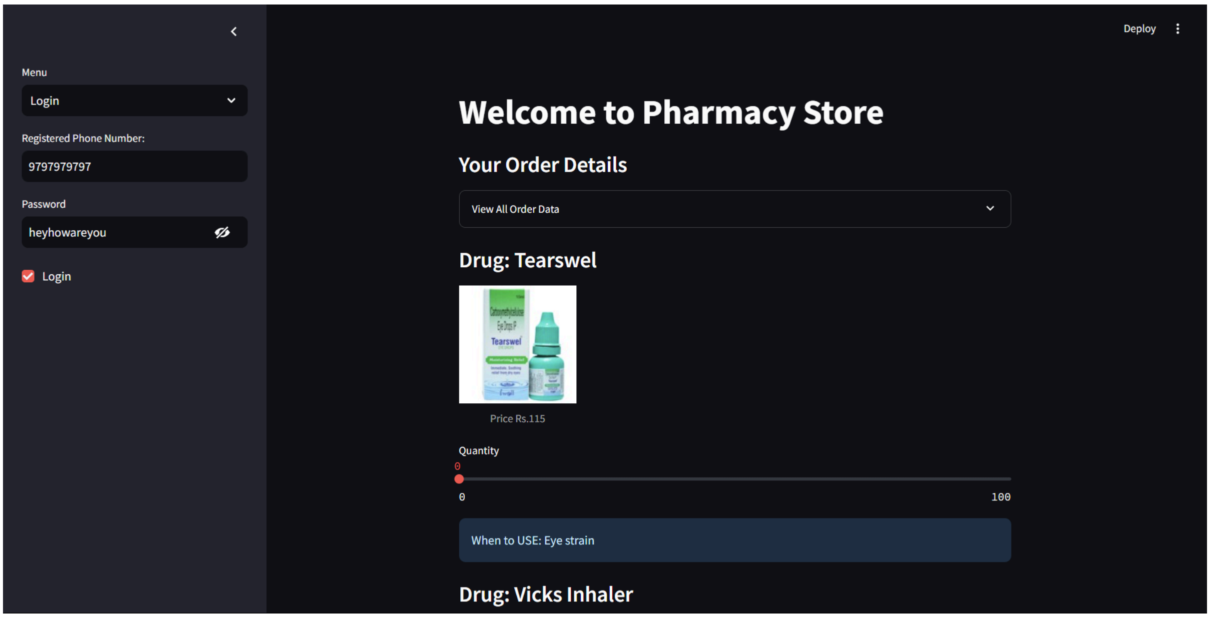


# Signing up as a new customer:

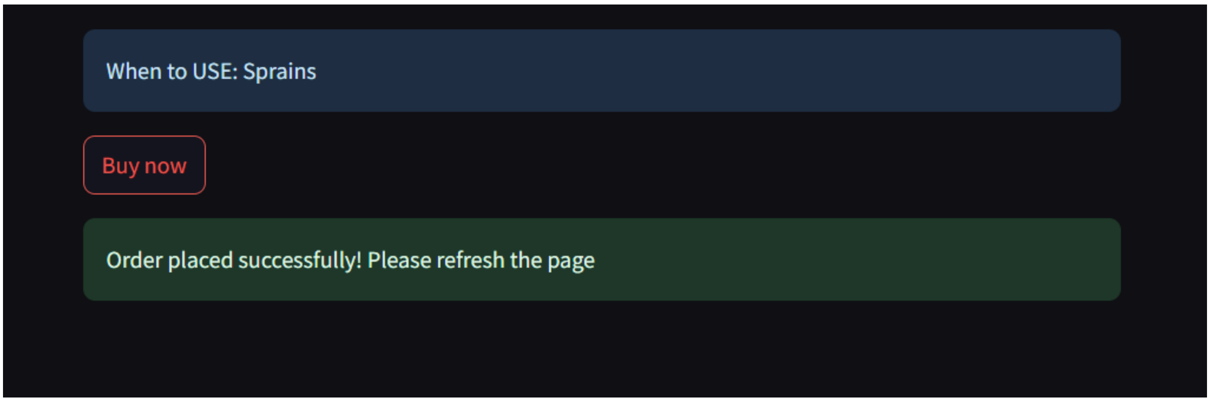


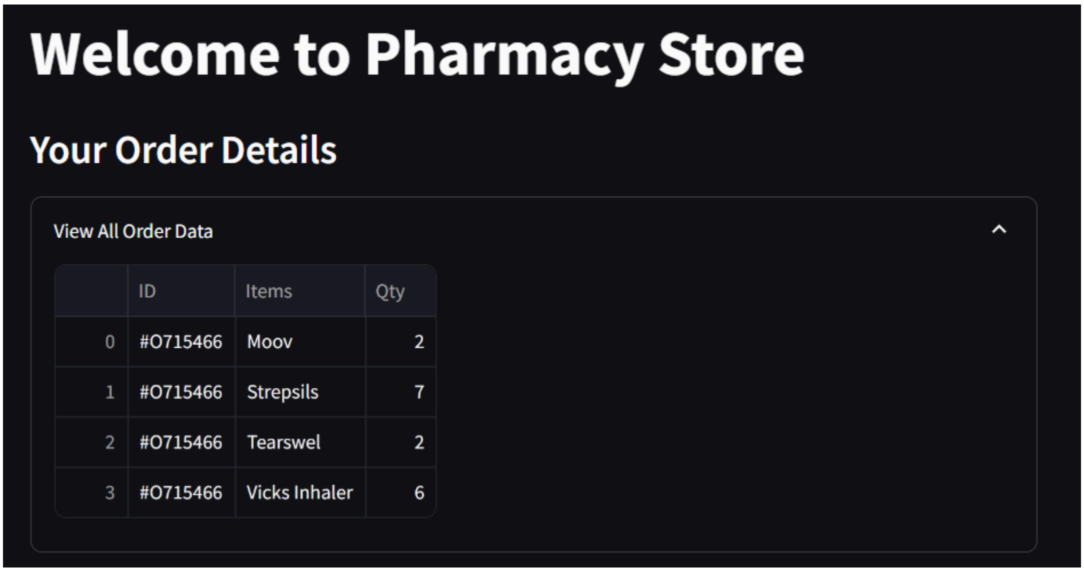
# Logging now:



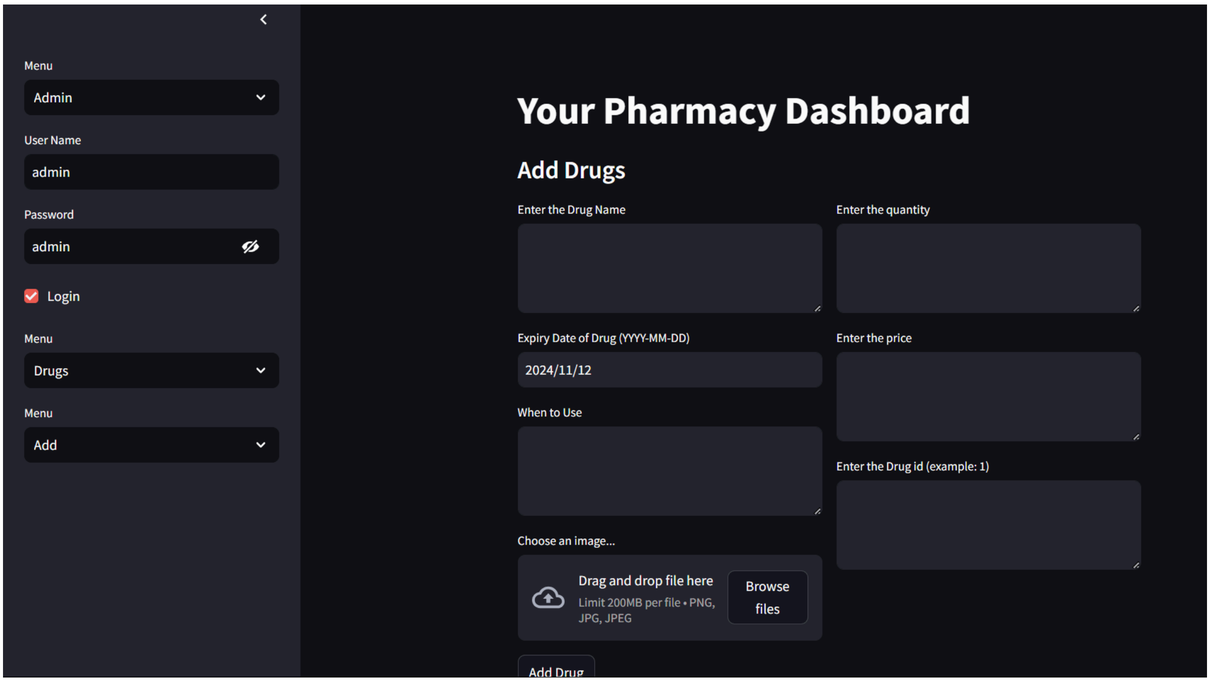

# Ordering some drugs:



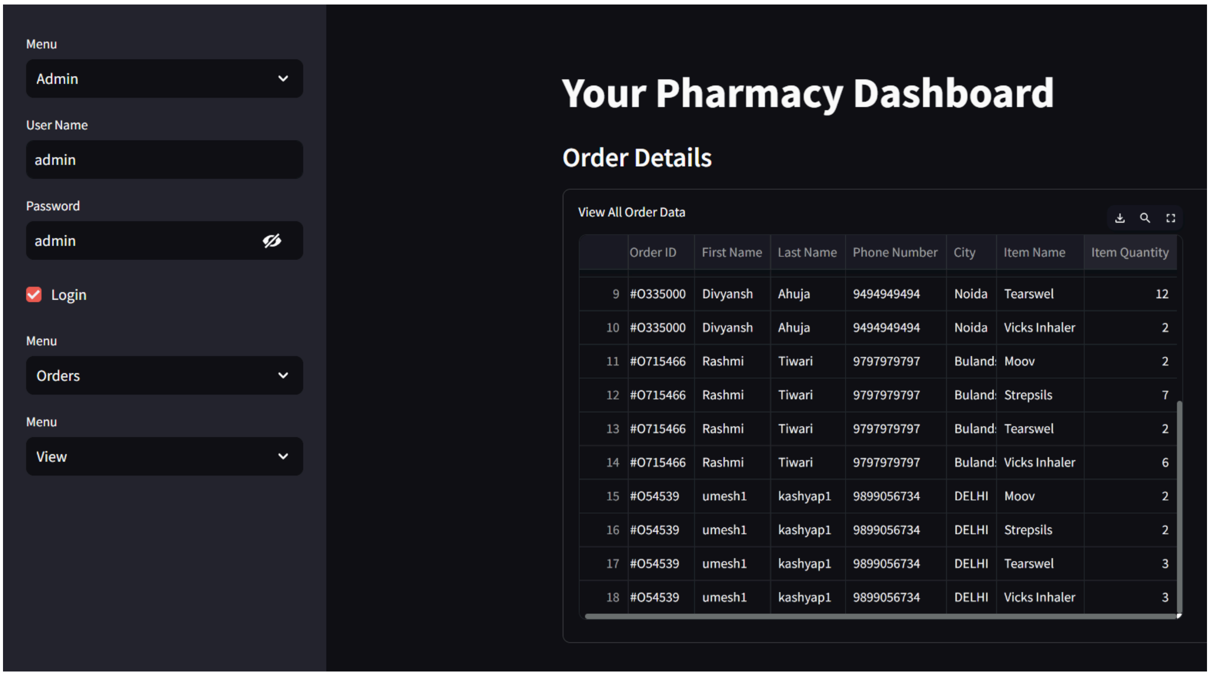
## Viewing order details:




## Admin dashboard:

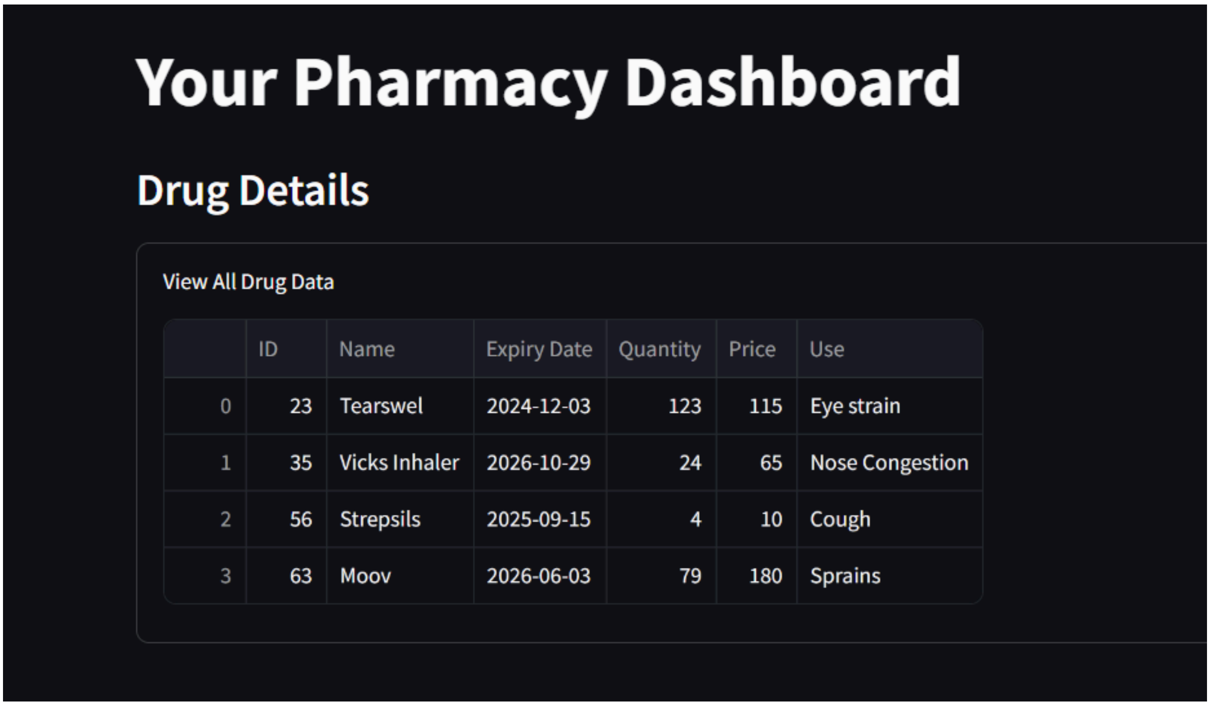


## Viewing order details: (Testing whether Rashmi's order received or not)



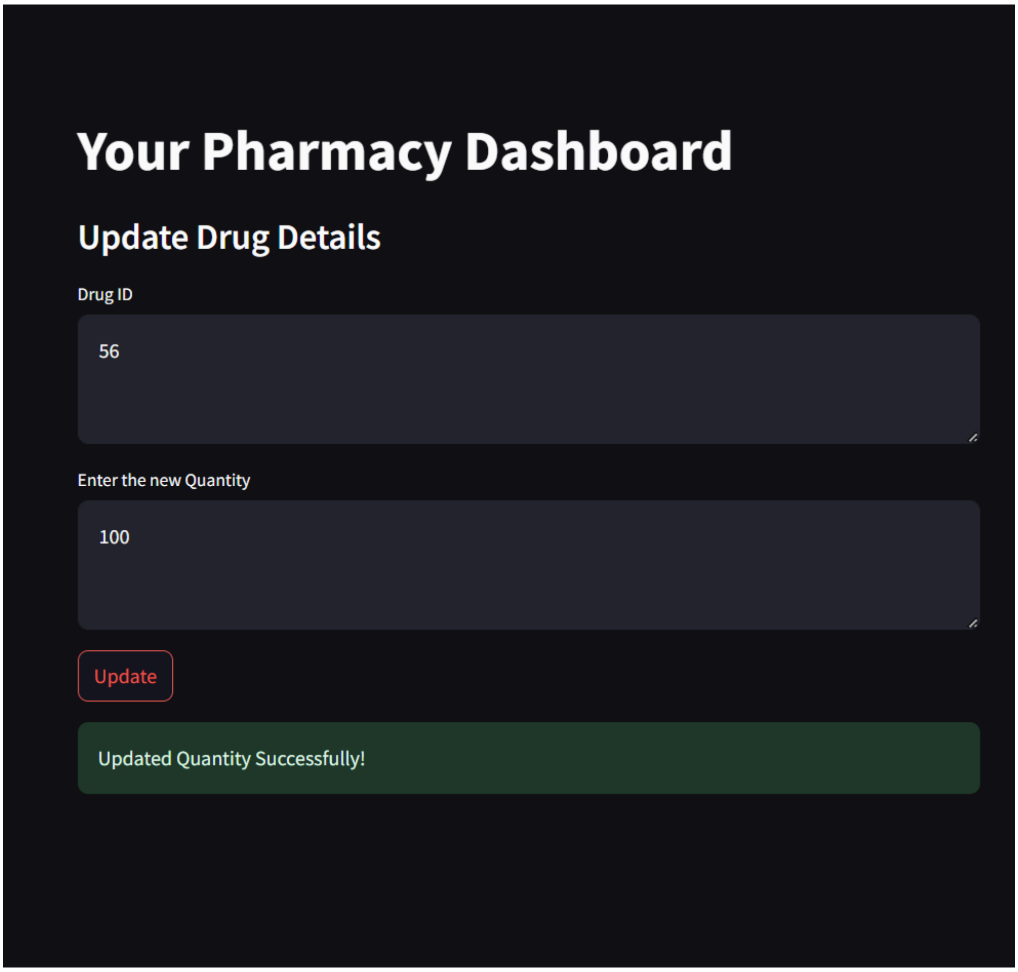

| | Order ID | First Name | Last Name | Phone Number | City | Item Name | Item Quantity |
|---|---|---|---|---|---|---|---|
| 9 | #O335000 | Divyansh | Ahuja | 9494949494 | Noida | Tearswel | 12 |
| 10 | #O335000 | Divyansh | Ahuja | 9494949494 | Noida | Vicks Inhaler | 2 |
| 11 | #O715466 | Rashmi | Tiwari | 9797979797 | Buland: | Moov | 2 |
| 12 | #O715466 | Rashmi | Tiwari | 9797979797 | Buland: | Strepsils | 7 |
| 13 | #O715466 | Rashmi | Tiwari | 9797979797 | Buland: | Tearswel | 2 |
| 14 | #O715466 | Rashmi | Tiwari | 9797979797 | Buland: | Vicks Inhaler | 6 |
| 15 | #O54539 | umesh1 | kashyap1 | 9899056734 | DELHI | Moov | 2 |
| 16 | #O54539 | umesh1 | kashyap1 | 9899056734 | DELHI | Strepsils | 2 |
| 17 | #O54539 | umesh1 | kashyap1 | 9899056734 | DELHI | Tearswel | 3 |
| 18 | #O54539 | umesh1 | kashyap1 | 9899056734 | DELHI | Vicks Inhaler | 3 |

## Order received, drug quantities for sale also updated automatically:

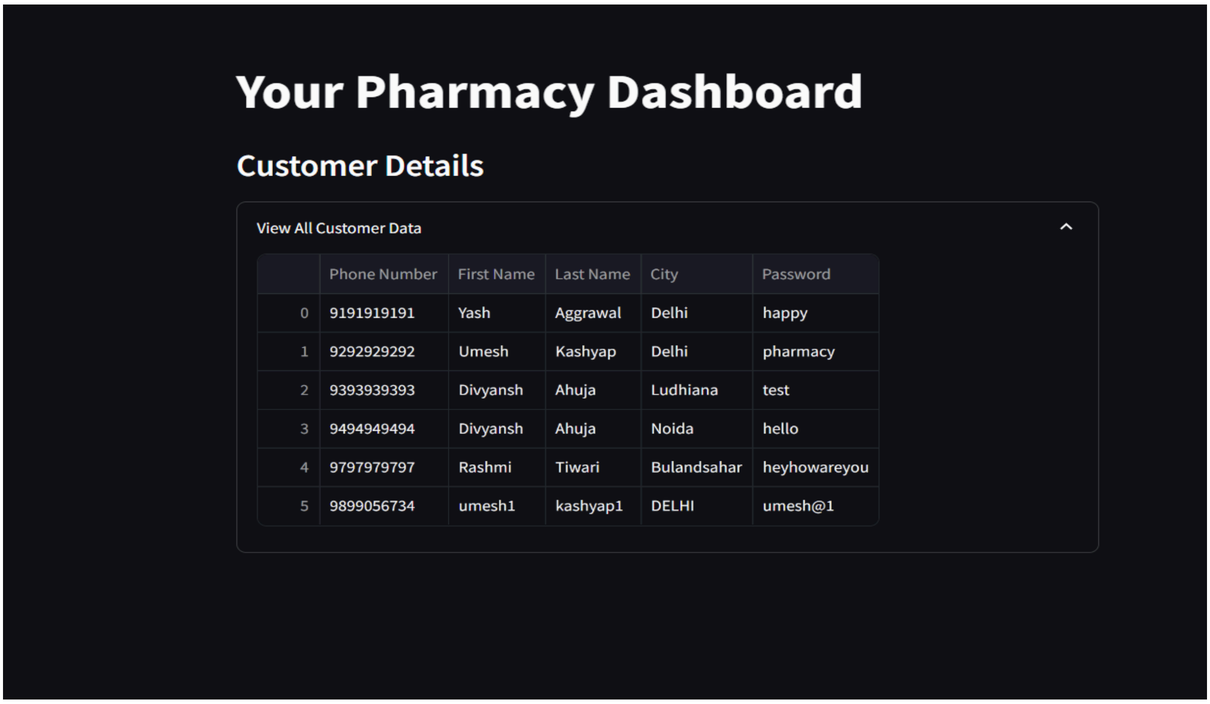



| | ID | Name | Expiry Date | Quantity | Price | Use |
|---|---|---|---|---|---|---|
| 0 | 23 | Tearswel | 2024-12-03 | 123 | 115 | Eye strain |
| 1 | 35 | Vicks Inhaler | 2026-10-29 | 24 | 65 | Nose Congestion |
| 2 | 56 | Strepsils | 2025-09-15 | 4 | 10 | Cough |
| 3 | 63 | Moov | 2026-06-03 | 79 | 180 | Sprains |

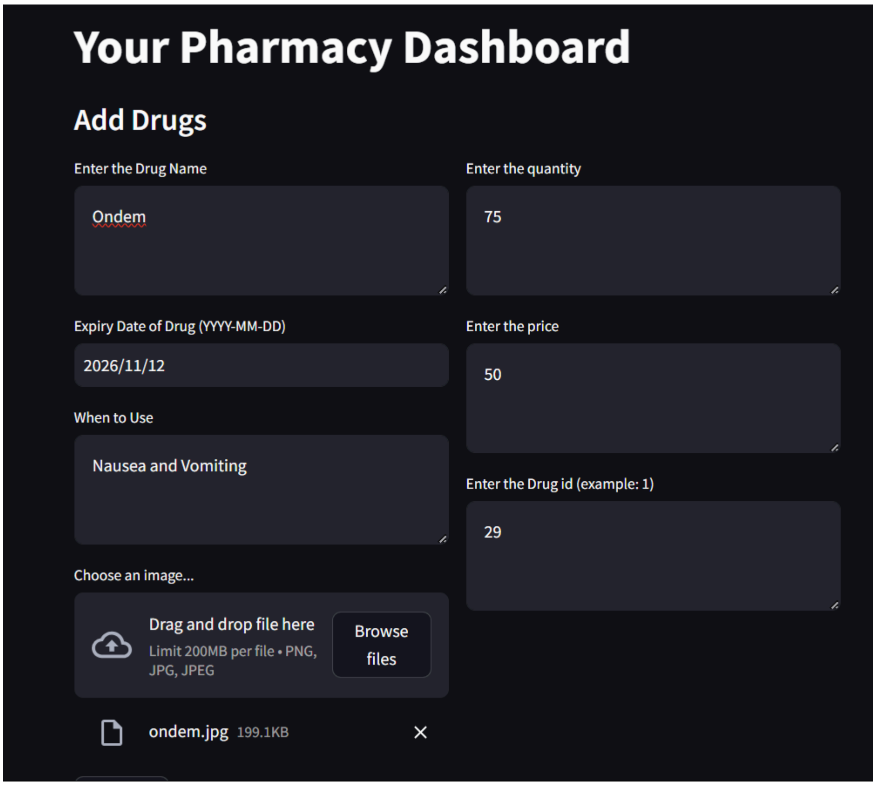
Updating Strepsils quantity to 100, since it has become low now:



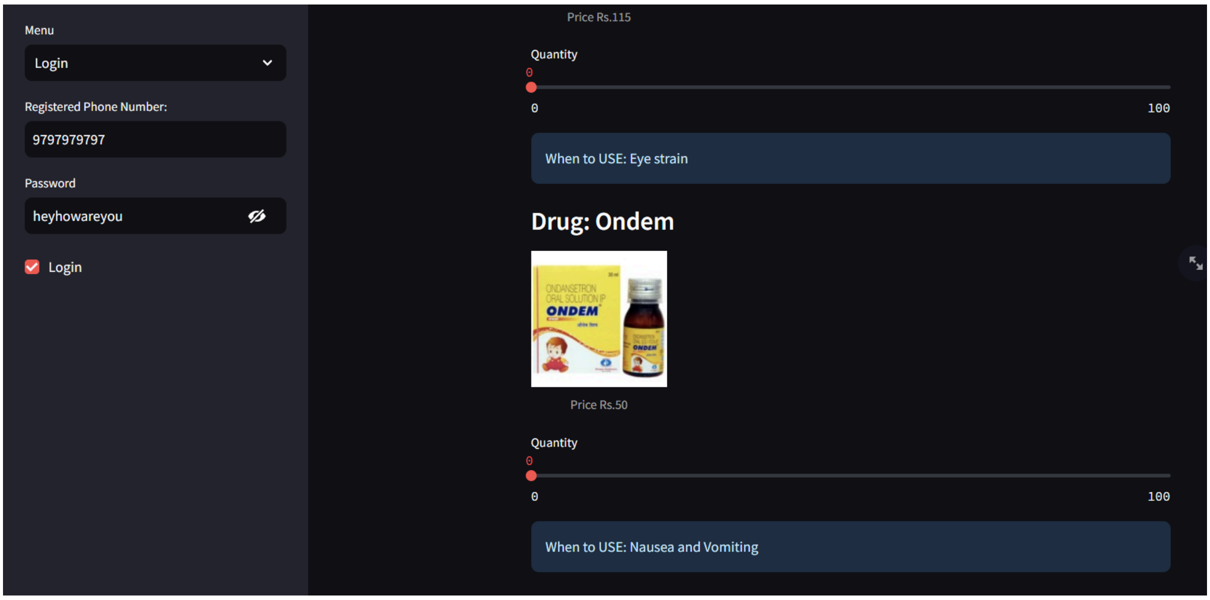
## Viewing customer data:




## Adding a new drug, ondem to the store:

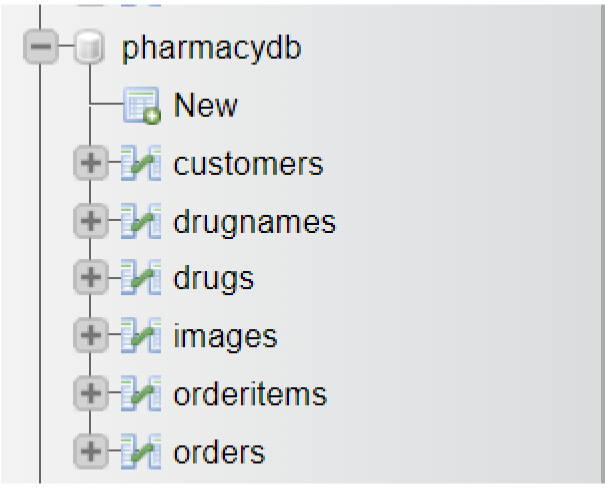


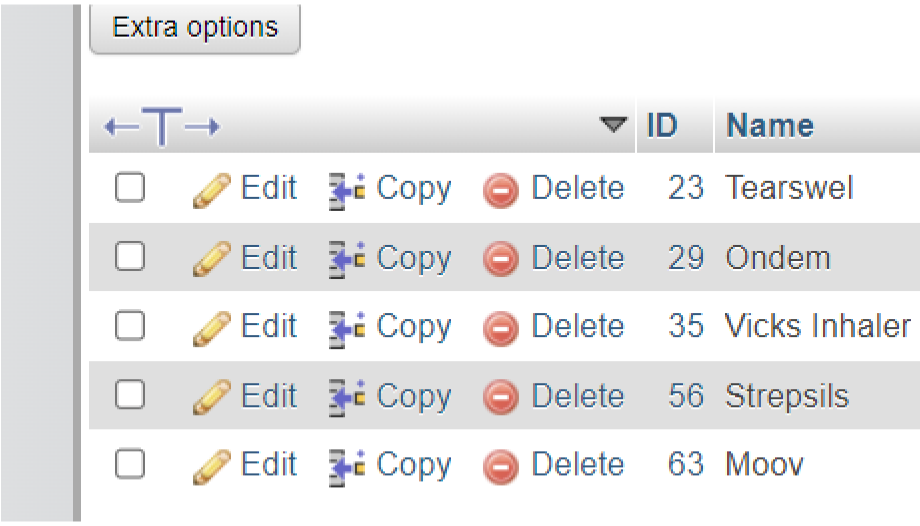

## Checking if it is now available for sale:




## Ondem now available to order!

## Database/Tables in XAMPP (Backend):



Extra options

| | | | | ID | Name |
|---|---|---|---|---|---|
| ☐ | Edit | Copy | Delete | 23 | Tearswel |
| ☐ | Edit | Copy | Delete | 29 | Ondem |
| ☐ | Edit | Copy | Delete | 35 | Vicks Inhaler |
| ☐ | Edit | Copy | Delete | 56 | Strepsils |
| ☐ | Edit | Copy | Delete | 63 | Moov |

| | | | | Phone | FirstName | LastName | City | Password |
|---|---|---|---|---|---|---|---|---|
| ☐ | Edit | Copy | Delete | 9191919191 | Yash | Aggrawal | Delhi | happy |
| ☐ | Edit | Copy | Delete | 9292929292 | Umesh | Kashyap | Delhi | pharmacy |
| ☐ | Edit | Copy | Delete | 9393939393 | Divyansh | Ahuja | Ludhiana | test |
| ☐ | Edit | Copy | Delete | 9494949494 | Divyansh | Ahuja | Noida | hello |
| ☐ | Edit | Copy | Delete | 9797979797 | Rashmi | Tiwari | Bulandsahar | heyhowareyou |
| ☐ | Edit | Copy | Delete | 9899056734 | umesh1 | kashyap1 | DELHI | umesh@1 |

| | | | | O_ID | PhoneNumber |
|---|---|---|---|---|---|
| ☐ | Edit | Copy | Delete | #O57242 | 9191919191 |
| ☐ | Edit | Copy | Delete | #O278278 | 9292929292 |
| ☐ | Edit | Copy | Delete | #O512331 | 9292929292 |
| ☐ | Edit | Copy | Delete | #O335000 | 9494949494 |
| ☐ | Edit | Copy | Delete | #O715466 | 9797979797 |
| ☐ | Edit | Copy | Delete | #O54539 | 9899056734 |

| | | | | ID | Image |
|---|---|---|---|---|---|
| ☐ | Edit | Copy | Delete | 23 | [BLOB - 51.8 KiB] |
| ☐ | Edit | Copy | Delete | 29 | [BLOB - 199.1 KiB] |
| ☐ | Edit | Copy | Delete | 35 | [BLOB - 15.0 KiB] |
| ☐ | Edit | Copy | Delete | 56 | [BLOB - 215.3 KiB] |
| ☐ | Edit | Copy | Delete | 63 | [BLOB - 56.6 KiB] |

| | | | | ID | ExpDate | Qty | Price | D_Use |
|---|---|---|---|---|---|---|---|---|
| ☐ | Edit | Copy | Delete | 23 | 2024-12-03 | 123 | 115 | Eye strain |
| ☐ | Edit | Copy | Delete | 29 | 2026-11-12 | 75 | 50 | Nausea and Vomiting |
| ☐ | Edit | Copy | Delete | 35 | 2026-10-29 | 24 | 65 | Nose Congestion |
| ☐ | Edit | Copy | Delete | 56 | 2025-09-15 | 100 | 10 | Cough |
| ☐ | Edit | Copy | Delete | 63 | 2026-06-03 | 79 | 180 | Sprains |

| | | | | O_ID | Item_Name | Item_Qty |
|---|---|---|---|---|---|---|
| ☐ | Edit | Copy | Delete | #O278278 | Moov | 5 |
| ☐ | Edit | Copy | Delete | #O335000 | Moov | 2 |
| ☐ | Edit | Copy | Delete | #O335000 | Strepsils | 5 |
| ☐ | Edit | Copy | Delete | #O335000 | Tearswel | 12 |
| ☐ | Edit | Copy | Delete | #O335000 | Vicks Inhaler | 2 |
| ☐ | Edit | Copy | Delete | #O512331 | | 0 |
| ☐ | Edit | Copy | Delete | #O54539 | Moov | 2 |
| ☐ | Edit | Copy | Delete | #O54539 | Strepsils | 2 |
| ☐ | Edit | Copy | Delete | #O54539 | Tearswel | 3 |
| ☐ | Edit | Copy | Delete | #O54539 | Vicks Inhaler | 3 |
| ☐ | Edit | Copy | Delete | #O57242 | Dolo 650 | 7 |
| ☐ | Edit | Copy | Delete | #O57242 | Moov | 32 |
| ☐ | Edit | Copy | Delete | #O57242 | Strepsils | 25 |
| ☐ | Edit | Copy | Delete | #O57242 | Tearswel | 5 |
| ☐ | Edit | Copy | Delete | #O57242 | Vicks Inhaler | 46 |
| ☐ | Edit | Copy | Delete | #O715466 | Moov | 2 |
| ☐ | Edit | Copy | Delete | #O715466 | Strepsils | 7 |
| ☐ | Edit | Copy | Delete | #O715466 | Tearswel | 2 |
| ☐ | Edit | Copy | Delete | #O715466 | Vicks Inhaler | 6 |

## Conclusion

### Summary

The Pharmacy Database Management System is a secure and user-friendly application designed to automate pharmacy operations It provides functionalities for both admins and customers and is built using modern technologies like Streamlit and MySQL.

### Future Enhancements

- Implementing a more complex reporting system.
- Adding advanced search functionality for drugs.
- Integration with third-party payment systems.

### Lessons Learned

- Building secure and scalable systems requires careful planning, especially when handling sensitive data like passwords and customer information.
- Streamlit is an excellent tool for quickly developing interactive web applications with minimal code.

---

## References


1. **MySQL Documentation**: https://dev.mysql.com/doc/
2. **Streamlit Documentation**: https://docs.streamlit.io/